

LHIC — 3-Minute Actual Demo

Recording runbook for an OpenAI Build Week submission

The demo should prove one clean boundary: Codex proposes typed work through LHIC MCP; LHIC performs local actions, asks for human authority, verifies outcomes, learns only from evidence, and later replays an independently promoted skill without an LLM or MCP.

What the judges must see

- Codex is visibly connected to the local lhic-computer-use MCP server.
- You are the live Slow Path operator: you enter the task, review the plan, approve activation, sign checkout, and decide whether execution continues.
- The browser is controlled through LHIC semantic actions—not Codex browser control, screenshots, OCR, page-evaluate JavaScript, or raw coordinates.
- The final purchase signature is entered by you. This is a deliberate human-authority moment; LHIC must not pretend it drew the signature.
- Verifier evidence and redacted trace/candidate metadata are shown in the LHIC Desktop App.
- The Fast Path evidence says 0 LLM calls / 0 MCP calls during local execution.
- Challenge2026 playback shows the local action loop separately from the website workflow.

Truth-in-demo rule

One successful Slow Path run creates a candidate; it does not become Fast Path immediately. LHIC requires three independent verified task IDs and a separate offline holdout with an unseen UI fingerprint. This runbook starts from a fresh memory database: the Slow Path builds the vendor candidate on camera, then the remaining real gates are shown in a clearly labelled compressed sequence before promotion.

Recommended spoken opening

“Codex will plan through LHIC MCP. LHIC—not the model—will execute, gate risky steps, verify every result, and build the candidate live. I’ll then complete the remaining evidence gates before the local replay.”

Use the four-slide deck only as a 5-second opener and a 7-second close. The browser, Codex task, and LHIC Desktop App are the presentation.

A 2:55 edit that still contains a real demo

Time	Screen	Action and narration
0:00–0:05	Slide 1	"Codex proposes; LHIC executes locally and proves the outcome."
0:05–0:15	Codex	Show /mcp or the MCP status: lhic-computer-use connected. State that no other browser-control tool is allowed.
0:15–1:25	Visible Chromium + Codex	Run the real vendor Slow Path. Keep approvals and verifier results at normal speed; accelerate only repetitive add-to-cart/fill footage to 1.5–2× with an on-screen speed label.
1:25–1:37	LHIC Desktop App	Show the new SLOW ONLY candidate at 1/3 with verifier evidence and redacted trace.
1:37–1:57	Compressed continuous evidence	Show real runs 2 and 3 plus the unseen-UI holdout at 4–8×. Keep task IDs, verifier pass, and holdout result readable at each endpoint.
1:57–2:20	LHIC Desktop App + Chromium	Promote only after the gates pass, then run the new Fast Path variant. Hold on FAST-READY, 0 LLM, 0 MCP, verifier passed.
2:20–2:45	Challenge2026	Play a 20-second local policy replay and show observed realtime metrics.
2:45–2:55	Slide 4	"Codex proposed the work. LHIC built the skill from proof, then executed locally."

Editing rules

- Do not cut across an approval prompt, verifier result, or the transition from Codex to LHIC.
- A speed-up is acceptable for repeated low-risk actions if the label remains visible and the action sequence is continuous.
- Never type demo codes while the recording zooms into the field. Password-field values and traces should remain redacted.
- Do not call the second run a different website. It is the same application and origin with a different account/state. LHIC intentionally will not generalize a skill to a new origin.

Pre-record setup and preflight

1. Build and connect the local runtime

```
cd /Users/chengmatt/Projects/ComputerIntent
npm ci
npm run pw:install
npm run build
export LHIC_MEMORY_DATABASE="$PWD/.lhic/build-week-live-learning.sqlite"
npm run mcp:config
```

Review the generated Codex configuration, restart Codex, and verify the server in /mcp. Keep the browser visible; do not set LHIC_MCP_HEADLESS.

2. Start the LHIC Desktop App

```
npm run desktop:start
```

Open Task Console and Skill Depot from the same shell environment so the MCP server and Desktop App use the fresh database above. Before recording, use read-only skill inspection to confirm there is no vendor candidate or promoted vendor skill in that database.

3. Prepare the vendor site

- Confirm <https://vendor.techtools.qzz.io/> and /finance are reachable.
- Use a fresh Chromium session. Rehearse the exact labels because the site is Chinese and may change.
- The product stock editor commits on blur; after filling “Test 庫存”, press Tab and wait for the refresh.
- The checkout requires a non-empty signature canvas. You must sign it manually before LHIC confirms the purchase.

4. Private rehearsal inputs—do not put these in slides or traces

Run	Store employee	Finance manager	Order	Finance/stock
Slow	LHICTEST	LHICMANAGER	test3 ×1; test2 ×2	支出 200; note 進貨; Test stock → 20
Fast variant	LHICTEST2	LHICMANAGER2	test3 ×2; test2 ×1	支出 201; note 進貨-Fast; Test stock → 21

The 200/201 amounts and fast variant are recommended rehearsal values because the request did not specify an expense amount or exact alternate quantities. Change them before recording if your test data requires another value.

5. Rehearse the live evidence-gate sequence

Do not seed or prebuild a vendor skill. During recording, the first completed batch creates the candidate at 1/3; runs 2 and 3 must use distinct task IDs; then a separate evaluator must run against the registered vendor test account or allowlisted sandbox with a genuinely unseen UI fingerprint. Promotion is permitted only when Skill Depot shows all gates passed. If the current build cannot run that evaluator from a user-facing surface, the honest video stops at SLOW ONLY and does not show vendor Fast Path.

Vendor Slow Path: the exact live sequence

You perform this section live

Do not run a prepared vendor demo script. In the recording, type the task into Codex yourself, provide the private values only when requested, review each displayed target, give the human approvals, and sign the checkout canvas. Codex may propose through MCP; LHIC alone performs and verifies browser actions.

Paste this prompt into Codex

Use only the connected LHIC MCP tools for browser work. Do not use Codex browser control, screenshots, OCR, page-evaluate JavaScript, raw coordinates, or another browser tool. Start a visible LHIC browser, observe before every target, execute semantic actions, and show verifier evidence after every action.

On <https://vendor.techtools.qzz.io/>, sign in with the employee code I will provide privately. Add test3 ×1 and test2 ×2. Open checkout, stop so I can sign the signature canvas manually, then ask for exact-action approval before confirming the order. Next navigate to /finance, sign in with the manager code I will provide privately, add a 支出 of 200 with note 進貨, then open 商品庫存 and set Test stock to 20. Press Tab after the stock fill and verify the refreshed row shows 20. Never print either code.

On-screen action order

Phase	What happens	Required evidence
Connect	lhic_runtime_status → lhic_browser_start → lhic_browser_observe	Visible Chromium and local runtime status
Store login	Fill the password field with LHICTEST; activate login	Authenticated storefront/catalog visible; value redacted
Cart	Click test3 once and test2 twice	Cart quantity after each click
Checkout	Open checkout; you manually sign the canvas	Checkout and non-empty signature visible
Purchase	LHIC activates the final confirm only after human approval	Success/receipt state and empty/updated cart
Finance login	Navigate /finance; fill LHICMANAGER; log in	財務與管理看板 visible; code redacted
Expense	Select 支出; fill 200 and 進貨; click 新增	New ledger row contains 支出, 200, 進貨
Stock	Open 商品庫存; fill the Test row's 庫存 input with 20; press Tab	Refreshed Test row shows 20

Learning evidence

A completed lhic_browser_execute_plan batch whose every step has evidence can record one candidate run. Individual lhic_browser_act calls can improve selector memory but do not create a Skill candidate. Batch execution pauses before activation steps: acquire real human approvals and resume—never fabricate approval JSON.

If the batch/approval rehearsal is too slow for the final edit, keep the real browser execution but show selector-memory learning only. Do not display a candidate counter that the run did not actually change.

Learning, verification, and honest Fast Path proof

What to show in the Desktop App

- Task status: completed, not merely “click dispatched”.
- Verifier/postcondition evidence for the order, expense row, and Test stock value.
- Trace/event entries with credential values omitted.
- Candidate status and verified-run count. Say the number exactly as displayed.
- After the on-camera evidence gates: FAST-READY and the execution summary with 0 LLM / 0 MCP.

Spoken bridge

“The Slow Path built this candidate live. I then completed two more independent verified runs and a separate unseen-UI holdout. Only now does LHIC permit promotion, so the next execution stays local.”

Fast variant

Use the same origin with a changed test account and changed values: LHICTEST2 / LHICMANAGER2, test3 ×2 and test2 ×1, 支出 201 with note 進貨-Fast, and Test stock 21. This proves controlled parameter/state variation, not arbitrary cross-site generalization.

Fast Path prompt in the Desktop Task Console

Run the approved vendor order-and-finance skill locally. Use the private employee and manager inputs I provide in the task UI. Order test3 ×2 and test2 ×1; pause for my checkout signature and exact-action approval; then add 支出 201 with note 進貨-Fast and set Test stock to 21. Require verifier evidence for the order result, ledger row, and final stock row.

Evidence shot—hold for two seconds

Frame the Desktop App so the judge can read: FAST-READY • local deterministic • model calls 0 • MCP calls 0 • verifier passed. If any field does not say that, stop and rerun; do not narrate around a contradictory screen.

What not to claim

- Do not say the one recorded run was instantly promoted.
- Do not say the second run is a different website; it is a different account/state on the same site.
- Do not call a Codex-compiled MCP batch “0 MCP”. Only the Desktop/local promoted skill replay is the strict 0-MCP Fast Path.

Challenge2026: separate local speed proof

Scope

This demo controls the player with W/A/S/D and Space. The game's trap-agent API is separate; do not imply LHIC calls it. No LLM, MCP, online multiplayer, score submission, or network service is used by the LHIC game-training path.

One-time setup

```
lhic train game env setup
lhic train game env doctor
lhic train game 2d setup challenge-2026 --source /Applications/Launcher.app/Contents/Resources/GameBuilds/Challenge2026.app
```

Open Challenge2026 yourself. Determine the exact active-window title and game-content rectangle. Replace the example values below; do not guess them.

Create a five-minute exact-target lease

```
lhic train game 2d lease challenge-2026 --window-title "Challenge2026" --region 100,100,1280,720 --approved-by local-operator --output /tmp/challenge-2026-lease.json
```

Record 20 seconds of your own play

```
lhic train game 2d record challenge-2026 --surface desktop --window-title "Challenge2026" --region 100,100,1280,720 --lease /tmp/challenge-2026-lease.json --duration-ms 20000 --output .lhic/game-training/2d/datasets/challenge-2026-human-v1
```

Fit the local policy

```
lhic train game 2d fit challenge-2026 --dataset .lhic/game-training/2d/datasets/challenge-2026-human-v1/manifest.json --seed 17 --validation-split 0.2 --output .lhic/game-training/2d/skills/challenge-2026-v1
```

Create a fresh lease immediately before playback, then run

```
lhic train game 2d play challenge-2026 --surface desktop --artifact .lhic/game-training/2d/skills/challenge-2026-v1/artifact.json --window-title "Challenge2026" --region 100,100,1280,720 --lease /tmp/challenge-2026-lease.json --duration-ms 25000
```

Show the returned realtime summary: requested control rate, observed rate, processing P50/P95, frame P95, and deadline misses. Say “configured local 20 Hz loop” unless the observed result also supports a stronger statement.

The lease expires after five minutes. Recreate it after fitting or any long rehearsal; focus loss releases all approved inputs.

Final rehearsal checklist

- The short PowerPoint opens and contains exactly four slides.
- Codex shows lhic-computer-use connected and no alternate browser-control tool is enabled for the task.
- Visible Chromium is controlled through LHIC semantic actions.
- The slow order uses test3 ×1 and test2 ×2 with LHICTEST.
- You manually sign checkout; the final submit shows a real approval boundary and verifier result.
- Finance uses LHICMANAGER, 支出 200, note 進貨, and Test stock 20; the stock fill is committed by blur/Tab.
- Desktop App evidence matches what you say: trace redacted, verifier passed, candidate count exact.
- The vendor skill did not exist before recording; after the shown gates it is FAST-READY and uses LHICTEST2/LHICMANAGER2 for the alternate run.
- The Fast Path summary visibly reports 0 LLM and 0 MCP calls.
- Challenge2026 uses the exact focused window and current lease; the policy controls only W/A/S/D/Space.
- The final video is ≤2:59, with speed labels on accelerated footage and no hidden approval/verifier cuts.

Failure fallbacks

Failure	What to do
Vendor label changed	Stop. Observe again and use the returned semantic target; do not guess a selector or use coordinates.
Signature blocks checkout	Sign manually. Keep this in the video as human authority.
Stock remains unchanged	Press Tab to trigger blur, wait for refresh, and verify the Test row before continuing.
Candidate does not increment	State that only selector memory changed if stepwise actions were used. Do not show false candidate learning.
Fast skill is not FAST-READY	Do not claim Fast Path and do not preload it. End the vendor section at SLOW ONLY or postpone recording until the real evidence gates pass.
Game stops	Refocus the exact window, recreate the lease, and rerun. Do not bypass focus protection.

Sources

Project: docs/architecture.md, docs/mcp-harnesses.md, docs/desktop-control-center.md, docs/game-training.md. Game rules and controls: <https://hackmd.io/@NDWuVyohSDi-kIZ4DgcUuQ/ry1gjR4Gze> and linked Game Introduction / Setup / Agent Development / API Reference pages.

Prepared for the current working tree on 22 July 2026. Keep this PDF private because it contains test-account codes supplied for rehearsal.